

Why can't constructors be virtual?

A virtual function relies on **runtime polymorphism**:

```
Base* p = new Derived();  
p->foo();    // calls Derived::foo()
```

At the time `foo()` is called, the object is **already constructed**, so C++ knows its runtime type and can use the vtable to select the correct function.

But a constructor's job is to **create/initialize the object in the first place**.

When constructing:

```
Derived d;
```

the object doesn't yet exist as a fully constructed `Derived`.

So asking:

```
virtual Derived();
```

would essentially mean:

"Before creating the object, dynamically determine which object's constructor should create it."

That's circular.

Interview-ready answer

Constructors cannot be virtual because virtual dispatch requires an already-created object, while a constructor is responsible for creating and initializing that object. Therefore, there is no object/vtable-based runtime dispatch available before construction.

What about destructors?

Destructors **can and often should be virtual**:

```
class Base {  
public:  
    virtual ~Base() = default;  
};
```

Because by the time destruction begins, the object **already exists**.

This is important:

```
Base* p = new Derived();  
delete p;
```

With a virtual destructor:

```
Derived destructor  
  ↓  
Base destructor
```

How do we achieve "virtual constructor" behavior?

You can't make the constructor virtual, but you can use a **factory function**.

For example:

```
class Animal {  
public:  
    virtual ~Animal() = default;  
  
    static unique_ptr<Animal> create(string type);  
};
```

Then:

```
auto animal = Animal::create("Dog");
```

The factory decides **which derived object to construct**.

This is commonly called the **Factory Pattern**.

Yes, syntactically it can:

```
class Base {
public:
    Base() {
        foo();
    }

    virtual void foo() {
        cout << "Base";
    }
};
```

But during construction, virtual dispatch **doesn't call the derived override**.

So:

```
Base constructor
↓
Base::foo()
```

not `Derived::foo()`.

Why?

Because the `Derived` portion hasn't been constructed yet.

Remember this

QuestionAnswerCan constructor be virtual? ☒ NoCan destructor be virtual? ☒ YesWhy constructor not virtual?Object must exist before virtual dispatchCan constructor call virtual function? ☒ Yes, but derived override won't dispatchAlternative to virtual constructor?Factory pattern

One-liner for interviews

```
"Constructors cannot be virtual because virtual dispatch requires an existing object, whereas a constructor is responsible for creating that object."
```